

Лабораторная работа № 1. Подготовка лабораторного стенда

1.1. Цель работы

Целью данной работы является приобретение практических навыков установки Rocky Linux на виртуальную машину с помощью инструмента Vagrant.

1.2. Предварительные сведения

Vagrant — представляет собой инструмент для создания и управления средами виртуальных машин в одном рабочем процессе.

Этот инструмент, по сути, позволяет автоматизировать процесс установки на виртуальную машину как основного дистрибутива операционной системы, так и настройки необходимого в дальнейшем программного обеспечения.

С проектом Vagrant и документацией по этому инструментальному средству можно ознакомиться на сайте <https://www.vagrantup.com>.

Основные понятия Vagrant:

- провайдер (provider) — система виртуализации, с которой работает Vagrant (например, VirtualBox, VMWare и т.п.);
- box-файл (или Vagrant Box) — сохранённый образ виртуальной машины с развёрнутой в ней операционной системой; по сути, box-файл используется как основа для клонирования виртуальных машин с теми или иными настройками;
- Vagrantfile — конфигурационный файл, написанный на языке Ruby, в котором указаны настройки запуска виртуальной машины.

1.2.1. HCL-файл настроек виртуальной машины

Packer использует язык конфигурации Hashicorp (HCL), разработанный для краткого описания необходимых шагов для доступа к файлу сборки. Язык HCL базируется на идеях языка JSON.

Это специальный файл с описанием метаданных по установке дистрибутива на виртуальную машину. HCL-файл является необязательным компонентом для создания box-файлов Vagrant, но полезен, так как позволяет управлять версиями и типами провайдеров (виртуального окружения) и образов операционных систем из одного файла.

1.2.2. Основные команды Vagrant

С Vagrant можно работать, используя следующие основные команды:

- `vagrant help` — вызов справки по командам Vagrant;
- `vagrant box list` — список подключённых к Vagrant box-файлов;
- `vagrant box add` — подключение box-файла к Vagrant;
- `vagrant destroy` — отключение box-файла от Vagrant и удаление его из виртуального окружения;
- `vagrant init` — создание «шаблонного» конфигурационного файла Vagrantfile для его последующего изменения;
- `vagrant up` — запуск виртуальной машины с использованием инструкций по запуску из конфигурационного файла Vagrantfile;
- `vagrant reload` — перезагрузка виртуальной машины;
- `vagrant halt` — остановка и выключение виртуальной машины;
- `vagrant provision` — настройка внутреннего окружения имеющейся виртуальной машины (например, добавление новых инструкций (скриптов) в ранее созданную виртуальную машину);
- `vagrant ssh` — подключение к виртуальной машине через ssh.

1.2.3. Пример конфигурации Vagrantfile

Приведём пример содержимого файла Vagrantfile для понимания принципов его синтаксиса:

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

Vagrant.configure("2") do |config|
  config.vm.box = "BOX_NAME"
  config.vm.hostname = "HOST_NAME"
  config.vm.network "private_network", ip: "192.168.1.1"
  config.vm.define "VM_NAME"
  config.vm.provider "virtualbox" do |vb|
    vb.gui = false
    vb.memory = "1024"
  end
end
```

Первые две строки указывают на режим работы с Vagrantfile и использование языка Ruby. Затем идёт цикл do, заменяющий конструкцию Vagrant.configure далее по тексту на config.

Строка config.vm.box = "BOX_NAME" задаёт название образа (box-файла) виртуальной машины (обычно выбирается из официального репозитория).

Строка config.vm.hostname = "HOST_NAME" задаёт имя виртуальной машины.

Конструкция config.vm.network задаёт тип сетевого соединения и может иметь следующие назначения:

- config.vm.network "private_network", ip: "xxx.xxx.xxx.xxx" — адрес из внутренней сети;
- config.vm.network "private_network", type: "dhcp" — адрес, назначаемый по протоколу DHCP.

Строка config.vm.define "VM_NAME" задаёт название виртуальной машины, по которому можно обращаться к ней из Vagrant и VirtualBox.

В конце идёт конструкция, определяющая параметры провайдера, а именно запуск виртуальной машины без графического интерфейса и с выделением 1 Гб памяти.

1.3. Сетевая топология

Внутренняя топология лабораторных использует сервер одновременно как маршрутизатор (рис. 1.1).

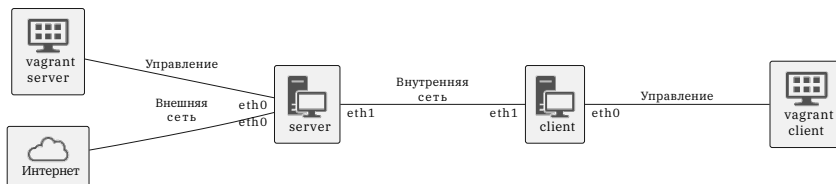


Рис. 1.1. Сетевая топология стенда

Настройка использования сервера как маршрутизатора проводится в лабораторной работе «Расширенные настройки межсетевого экрана». Без этой настройки клиент будет лишён доступа в Интернет.

Кроме того, сервер выступает также в роли сервера адресов DHCP, настройка которого производится в лабораторной работе «Настройка DHCP-сервера». Без этой настройки клиент не будет получать ip-адрес внутренней сети.

1.4. Задание

1. Сформируйте box-файл с дистрибутивом Rocky Linux для VirtualBox (см. раздел 1.5.2 или 1.5.3).
2. Запустите виртуальные машины сервера и клиента и убедитесь в их работоспособности.
3. Внесите изменения в настройки загрузки образов виртуальных машин `server` и `client`, добавив пользователя с правами администратора и изменив названия хостов (см. раздел 1.5.4).
4. Скопируйте необходимые для работы с Vagrant файлы и box-файлы виртуальных машин на внешний носитель. Используя эти файлы, вы можете попробовать вернуть виртуальные машины на другом компьютере.

1.5. Последовательность выполнения работы

Подготовленный box-файл Vagrant впоследствии можно использовать как в ОС Linux (в дисплейном классе или на собственном компьютере), так и в ОС Windows (только на собственном компьютере). Если вы планируете работать на собственном компьютере, то убедитесь, что:

- у вас достаточно свободного места на диске в разделе, где будет разворачиваться образ виртуальной машины (рекомендуется зарезервировать порядка 15GB);
 - в вашей операционной системе установлены последние версии Vagrant (<https://www.vagrantup.com>) и VirtualBox (<https://www.virtualbox.org/>);
 - для ОС Windows понадобится дополнительно установить Packer (<https://www.packer.io/>) и FAR (<https://www.farmanager.com>) для удобства работы в терминале.
- Далее приведена последовательность действий по подготовке box-файл Vagrant.
1. Перед началом работы с Vagrant создайте каталог для проекта.
В ОС Linux рекомендуется работать в `/var/tmp`:

```
mkdir -p /var/tmp/user_name/packer
mkdir -p /var/tmp/user_name/vagrant
```

где `user_name` — идентифицирующее вас имя пользователя, обычно первые буквы инициалов и фамилия.
В ОС Windows, например, `C:\work\user_name\packer` и `C:\work\user_name\vagrant`, где `user_name` — идентифицирующее вас имя пользователя, обычно первые буквы инициалов и фамилия.
 2. В созданном рабочем каталоге в подкаталоге `packer` разместите образ варианта операционной системы Rocky Linux (в этом практикуме будем использовать Rocky-10.0-x86_64-minimal.iso — минимальный дистрибутив Rocky Linux, который можно взять с сайта <https://rockylinux.org/download/>). При работе в дисплейном классе университета дистрибутив можно взять из общего каталога `/afs/dk.sci.pfu.edu.ru/common/files/iso/`.
 3. В этом же рабочем каталоге разместите подготовленные заранее для работы с Vagrant файлы:
 - в подкаталоге `packer` файл `vagrant-rocky.pkr.hcl` — специальный файл с описанием метаданных по установке дистрибутива на виртуальную машину (содержание используемого в данном практикуме файла `.hcl` приведено в разделе 1.5.1.1); в частности, в разделе переменных этот файл содержит указание на версию дистрибутива, его хэш-функцию, имя и пароль пользователя по умолчанию; в разделе `builders` указаны специальные синтаксические конструкции для автоматизации работы VirtualBox; в разделе `provisioners` прописаны действия (по сути shell-скрипт) по установке дополнительных пакетов дистрибутива;
 - в подкаталоге `packer` подкаталог `http` с файлом `ks.cfg` — определяет настройки для установки дистрибутива, которые пользователь обычно вводит вручную, в частности настройки языка интерфейса, языковые настройки клавиатуры, тайм-зону, сетевые настройки и т.п.; файл `ks.cfg` должен быть расположен в подкаталоге `http` (содержание используемого в данном практикуме файла `./http/ks.cfg` приведено в разделе 1.5.1.2);

- в подкаталоге `vagrant` файл `Vagrantfile` — файл с конфигурацией запуска виртуальных машин — сервера и клиента (содержание используемого в данном практикуме на данном этапе файла `Vagrantfile` приведено в разделе 1.5.1.3);
- в подкаталоге `vagrant` файл `Makefile` — набор инструкций для программы `make` по работе с `Vagrant` (содержание используемого в данном практикуме файла `Makefile` приведено в разделе 1.5.1.5).

Основное назначение `Makefile` в этом практикуме — применение команд `Vagrant` в ОС Linux в определённом каталоге — только в каталоге с проектом (в частности в `/var/tmp/user_name/vagrant`). Для пользователей, работающих с `Vagrant` в ОС Windows, `Makefile` не понадобится.

- В этом же рабочем каталоге в подкаталоге `vagrant` создайте каталог `provision` с подкаталогами `default`, `server` и `client`, в которых будут размещаться скрипты, изменяющие настройки внутреннего окружения базового (общего) образа виртуальной машины, сервера или клиента соответственно.
- В каталогах `default`, `server` и `client` разместите заранее подготовленный скрипт-заглушку `01-dummy.sh` следующего содержания:

```
#!/bin/bash
echo "Provisioning script $0"
```

- В каталоге `default` разместите заранее подготовленный скрипт `01-user.sh` по изменению названия виртуальной машины следующего содержания:

```
#!/bin/bash

echo "Provisioning script $0"

username=user
userpassword=123456

encpassword=`openssl passwd -1 ${userpassword}`
id -u $username
if [[ $? ]]
then
    adduser -G wheel -p ${encpassword} ${username}
    homedir=`getent passwd ${username} | cut -d: -f6`
    echo "export PS1='[\u@\H \W]\\$ '" >> ${homedir}/.bashrc
fi
```

В этом скрипте в качестве значения переменной `username` вместо `user` укажите имя пользователя, совпадающее с вашим логином, т.е. для Ивана Петровича Сидорова логин должен иметь вид `ipsidorov`.

- В каталоге `default` разместите заранее подготовленный скрипт `01-hostname.sh` по изменению названия виртуальной машины следующего содержания:

```
#!/bin/bash

username=user
hostnamectl set-hostname "${HOSTNAME%.*}.${username}.net"
```

В этом скрипте в качестве значения переменной `username` вместо `user` укажите имя пользователя, совпадающее с вашим логином, т.е. для Ивана Петровича Сидорова логин должен иметь вид `ipsidorov`.

- В каталоге `server` разместите заранее подготовленный скрипт `02-forward.sh` следующего содержания:

```
#!/bin/bash

echo "Provisioning script $0"

echo "Enable forwarding"
echo "net.ipv4.ip_forward = 1" > /etc/sysctl.d/90-forward.conf
sysctl -w net.ipv4.ip_forward=1
```

```
echo "Configure masquerading"
firewall-cmd --add-masquerade --permanent
firewall-cmd --reload
```

```
restorecon -vR /etc
```

Этот скрипт обеспечивает корректную маршрутизацию ip-адресов между сервером и клиентом.

9. В каталоге client разместите заранее подготовленный скрипт 01-routing.sh следующего содержания:

```
#!/bin/bash
```

```
echo "Provisioning script $0"
```

```
nmcli connection modify "System eth1" ipv4.gateway "192.168.1.1"
nmcli connection up "System eth1"
```

```
nmcli connection modify eth0 ipv4.never-default true
nmcli connection modify eth0 ipv6.never-default true
```

```
nmcli connection down eth0
nmcli connection up eth0
```

Этот скрипт обеспечивает корректную работу сетевых интерфейсов клиента.

1.5.1. Конфигурационные файлы

1.5.1.1. Содержание файла vagrant-rocky.pkr.hcl

```
packer {
  required_plugins {
    vagrant = {
      source = "github.com/hashicorp/vagrant"
      version = "~> 1"
    }
    virtualbox = {
      version = "~> 1"
      source = "github.com/hashicorp/virtualbox"
    }
    qemu = {
      version = "~> 1"
      source = "github.com/hashicorp/qemu"
    }
  }
}

variable "artifact_description" {
  type = string
  default = "Rocky 10.2"
}

variable "artifact_version" {
  type = string
  default = "10.2"
}

variable "disk_size" {
  type = string
  default = "61440"
```

```

}

variable "iso_checksum" {
  type    = String
  default = "aac6ac3ce781b91a91ce78463405f66c611a5dca4b3840c79e5e01d97302f6c8"
}

variable "iso_checksum_type" {
  type    = String
  default = "sha256"
}

variable "iso_url" {
  type    = String
  # default = "https://download.rockylinux.org/pub/rocky/10/isos/x86_64/Rocky_
  ~ -10.2-x86_64-minimal.iso"
  default = "Rocky-10.2-x86_64-minimal.iso"
}

variable "redhat_platform" {
  type    = String
  default = "x86_64"
}

variable "redhat_release" {
  type    = String
  default = "10"
}

variable "ssh_password" {
  type    = String
  default = "vagrant"
}

variable "ssh_username" {
  type    = String
  default = "vagrant"
}

variable "http_directory" {
  type    = String
  default = "http"
}

source "qemu" "rockylinux" {
  boot_command = [
    "<up>",
    "e",
    "<down><down><end><wait>",
    " inst.ks=http://{{ .HTTPIP }}:{{ .HTTPPort }}/rocky10-ks.cfg ",
    " biosdevname=0 net.ifnames=0 ",
    "<enter><wait><leftCtrlOn>x<leftCtrlOff>"
  ]
  boot_wait = "10s"
  disk_size = "${var.disk_size}"
  http_directory = "${path.root}/${var.http_directory}"
  iso_checksum = "${var.iso_checksum_type}:${var.iso_checksum}"
  iso_url = "${var.iso_url}"
  output_directory = "output-rockylinux${var.redhat_release}-qemu"
}

```

```

format                                [] "qcow2"
ssh_password                          [] "${var.ssh_password}"
ssh_username                          [] "${var.ssh_username}"
ssh_timeout                           [] "60m"
vm_name                               [] "rockylinux${var.redhat_release}-qemu"
net_device                            [] "virtio-net"
disk_interface                         [] "virtio"
# headless                            [] true

## Настройки QEMU
## Параметры процессора
cpus                                  [] 2
memory                               [] 2048
accelerator                           [] "kvm"
cpu_model                             [] "host"                ## Использовать характеристики
# хоста
machine_type                          [] "q35"                ## Тип системной платы
firmware                              [] "/usr/share/edk2-ovmf/OVMF_CODE.fd" ## UEFI вместо
# BIOS. Проверьте местоположение

## Настройки видео
vga                                   [] "virtio" ## Для virtio-vga

## Дополнительные флаги процессора
qemuargs [] [
    ["-device", "qemu-xhci"], ## Виртуализированные USB-контроллеры
    ["-device", "virtio-tablet"], ## Устройства ввода
    ## GPU-passthrough
    ["-device", "virtio-gpu-pci"], ## 3D-акселерация через VirGL
    ["-vga", "none"]
]
}

source "virtualbox-iso" "rockylinux" {
    boot_command [] [
        "<up>",
        "e",
        "<down><down><end><wait>",
        " inst.ks=http://{{ .HTTPIP }}:{{ .HTTPPort }}/rocky10-ks.cfg ",
        " biosdevname=0 net.ifnames=0 ",
        "<enter><wait><leftCtrlOn>x<leftCtrlOff>"
    ]
    boot_wait                          [] "10s"
    disk_size                           [] "${var.disk_size}"
    export_opts                         [] [
        "--manifest",
        "--vsys", "0",
        "--description", "${var.artifact_description}",
        "--version", "${var.artifact_version}"
    ]
}

guest_additions_path                  [] "VBoxGuestAdditions.iso"
guest_os_type                         [] "RedHat_64"
http_directory                        [] "${var.http_directory}"
iso_checksum                          [] "${var.iso_checksum_type}:${var.iso_checksum}"
iso_url                               [] "${var.iso_url}"
output_directory                      []
# "output-rockylinux${var.redhat_release}-virtualbox"
shutdown_command                      [] "sudo -S /sbin/halt -h -p"
shutdown_timeout                      [] "5m"

```

```

ssh_password      "${var.ssh_password}"
ssh_username      "${var.ssh_username}"
ssh_port          22
ssh_ptty          true
ssh_timeout       "60m"
iso_interface     "sata"
headless          true
vboxmanage        [
  ["modifyvm", "${Name}", "--memory", "2048" ],
  ["modifyvm", "${Name}", "--cpus", "2" ],
  ["modifyvm", "${Name}", "--nat-localhostreachble1", "on" ],
  ["modifyvm", "${Name}", "--firmware", "EFI" ],
  ["modifyvm", "${Name}", "--vrde", "on"],
  ["modifyvm", "${Name}", "--vrdeport", "3390"]
]
virtualbox_version_file = ".vbox_version"
vm_name           = "rockylinux${var.redhat_release}-virtualbox"
}

build {
  sources = [
    "source.virtualbox-iso.rockylinux",
    "source.qemu.rockylinux"
  ]

  provisioner "shell" {
    execute_command = "echo 'packer'|${ .Vars }} sudo -S -E bash '${ .Path}
    scripts = ["scripts/vagrant.sh", "scripts/software.sh"]
  }

  provisioner "shell" {
    only = ["virtualbox-iso.rockylinux"]
    script = "scripts/virtualbox.sh"
  }

  provisioner "shell" {
    script = "scripts/cleanup.sh"
  }

  post_processor "vagrant" {
    compression_level = "6"
    output = "vagrant-${ .Provider}
    <  }}-rockylinux${var.redhat_release}-${var.redhat_platform}.box"
  }
}

```

1.5.1.2. Содержание файла rocky10-ks.cfg

```

# System bootloader configuration
bootloader --append="no_timer_check console=tty0 console=ttyS0,115200n8
<  net.ifnames=0 biosdevname=0 elevator=noop" --location=mbr --timeout=1
# Clear the Master Boot Record
zerombr
# Partition clearing information
clearpart --all
# Reboot after installation
reboot
# Use text mode install

```

```
text
# Keyboard layouts
keyboard --vckeymap=us,ru --xlayouts='us,ru'
# System language
lang en_US.UTF-8

# Network information
network --bootproto=dhcp --device=link --activate

# System authorization information
authselect select sssd with-sudo with-mkhomedir --force
authselect apply-changes
# Root password
rootpw vagrant
user --name=vagrant --password=vagrant
firstboot --disable
# Do not configure the X Window System
#skipx
# System services
services --enabled="NetworkManager,sshd,chrony"
# System timezone
timezone UTC --utc
user --name=vagrant --password=vagrant
# Disk partitioning information
# part / --fstype="xfs" --size=10239
bootloader --location=mbr
clearpart --all --initlabel
autopart --type=lvm

%post
# configure swap to a file
# fallocate -l 2G /swapfile
# chmod 600 /swapfile
# mkswap /swapfile
# echo "/swapfile none swap defaults 0 0" >> /etc/fstab

# sudo
echo "%vagrant ALL=(ALL) NOPASSWD: ALL" > /etc/sudoers.d/vagrant
chmod 0440 /etc/sudoers.d/vagrant

# Fix for https://github.com/CentOS/sig-cloud-instance-build/issues/38
cat > /etc/sysconfig/network-scripts/ifcfg-eth0 << EOF
DEVICE="eth0"
BOOTPROTO="dhcp"
ONBOOT="yes"
TYPE="Ethernet"
PERSISTENT_DHCLIENT="yes"
EOF

# sshd: disable password authentication and DNS checks
#ex -s /etc/ssh/ssh_config <<EOF
#:%substitute/^(PasswordAuthentication\) yes$/\1 no/
#:%substitute/^(UseDNS\) yes$/&\r\1 no/
#:%update
#:%quit
#EOF
#cat >>/etc/sysconfig/ssh <<EOF

# Decrease connection time by preventing reverse DNS lookups
# (see https://lists.centos.org/pipermail/centos-devel/2016-July/014981.html)
```

```

# and man sshd for more information)
OPTIONS="-u0"
EOF

# Fix for issue #76, regular users can gain admin privileges via su
ex -s /etc/pam.d/su <<'EOF'
# allow vagrant to use su, but prevent others from becoming root or vagrant
/^account\s\+sufficient\s\+pam_succeed_if.so uid = 0 use_uid quiet$/
:append
account          [success=1 default=ignore] \\\
                  pam_succeed_if.so user = vagrant use_uid quiet
account          required          pam_succeed_if.so user notin root:vagrant

:update
:quit
EOF

# systemd should generate a new machine id during the first boot, to
# avoid having multiple Vagrant instances with the same id in the local
# network. /etc/machine-id should be empty, but it must exist to prevent
# boot errors (e.g. systemd-journald failing to start).
:>/etc/machine-id

#echo 'vag' > /etc/yum/vars/infra

# Blacklist the floppy module to avoid probing timeouts
echo blacklist floppy > /etc/modprobe.d/nofloppy.conf
chcon -u system_u -r object_r -t modules_conf_t /etc/modprobe.d/nofloppy.conf

# Customize the initramfs
pushd /etc/dracut.conf.d
# There's no floppy controller, but probing for it generates timeouts
echo 'omit_drivers+=" floppy "' > nofloppy.conf
popd
# Fix the SELinux context of the new files
restorecon -f - <<EOF
/etc/sudoers.d/vagrant
#/etc/dracut.conf.d/vmware-fusion-drivers.conf
#/etc/dracut.conf.d/hyperv-drivers.conf
#/etc/dracut.conf.d/nofloppy.conf
EOF

# Rerun dracut for the installed kernel (not the running kernel):
KERNEL_VERSION=$(rpm -q kernel --qf '%{version}-%{release} %{arch}\n')
dracut -f /boot/initramfs-`${KERNEL_VERSION}`.img `${KERNEL_VERSION}`

# Seal for deployment
rm -rf /etc/ssh/ssh_host_*
hostnamectl set-hostname localhost.localdomain
rm -rf /etc/udev/rules.d/70-*
%end

%packages --inst-langs=en
@core
openssh-server
sudo
%end

%addon com_redhat_kdump --disable --reserve-mb='128'
%end

```

1.5.1.3. Содержание файла Vagrantfile

```
# -*- mode: ruby -*-
# vi: set ft=ruby :
```

```
Vagrant.configure("2") do |config|

  config.vagrant.plugins = ["vagrant-libvirt"]
  config.vagrant.plugins = ["vagrant-vbguest"]

  config.vm.provider :virtualbox do |virtualbox|
    virtualbox.linked_clone = true
    # Customize the amount of memory on the VM
    virtualbox.memory = 2048
    virtualbox.cpus = 2
    ## Display the VirtualBox GUI when booting the machine
    virtualbox.gui = false
    ## Set the video memory to 12Mb
    virtualbox.customize ["modifyvm", :id, "--vram", "32"]
    virtualbox.customize ["modifyvm", :id, "--natdnshostresolver1", "on"]
    virtualbox.customize ["modifyvm", :id, "--clipboard", "bidirectional"]
    virtualbox.customize ["modifyvm", :id, "--draganddrop", "bidirectional"]
    virtualbox.customize ["modifyvm", :id, "--graphicscontroller", "vmsvga"]
    virtualbox.customize ["modifyvm", :id, "--accelerate3d", "on"]
    virtualbox.customize ["modifyvm", :id, "--nested-hw-virt", "on"]
  end

  config.vm.provider :libvirt do |libvirt|
    libvirt.driver = "kvm"
    libvirt.memory = 2048
    libvirt.cpus = 2
    libvirt.video_type = "virtio"
    libvirt.disk_bus = "virtio"
    libvirt.nic_model_type = "virtio"
    libvirt.management_network_name = "vagrant-libvirt"
    libvirt.management_network_address = "192.168.121.0/24"
    libvirt.storage_pool_name = "vagrant"
    # libvirt.storage_pool_name = "default"
  end

  ## Common configuration
  config.vm.provision "common dummy",
    type: "shell",
    preserve_order: true,
    path: "provision/default/01-dummy.sh"

  config.vm.provision "common hostname",
    type: "shell",
    preserve_order: true,
    run: "always",
    path: "provision/default/01-hostname.sh"

  config.vm.provision "common user",
    type: "shell",
    preserve_order: true,
    path: "provision/default/01-user.sh"

  ## Server configuration
  config.vm.define "server", autostart: false do |server|
    server.vm.box = "rockylinux10"
  end
end
```

```

server.vm.hostname = 'server'

server.vm.boot_timeout = 1440

server.ssh.insert_key = false
server.ssh.username = 'vagrant'
server.ssh.password = 'vagrant'

server.vm.network :private_network,
                  ip: "192.168.1.1",
                  virtualbox__intnet: true

server.vm.provider :virtualbox do |virtualbox|
  virtualbox.customize ["modifyvm", :id, "--vrd", "on"]
  virtualbox.customize ["modifyvm", :id, "--vrdeport", "3391"]
end

server.vm.provision "server dummy",
                  type: "shell",
                  preserve_order: true,
                  path: "provision/server/01-dummy.sh"

end

## Client configuration
config.vm.define "client", autostart: false do |client|
  client.vm.box = "rockylinux10"
  client.vm.hostname = 'client'

  client.vm.boot_timeout = 1440

  client.ssh.insert_key = false
  client.ssh.username = 'vagrant'
  client.ssh.password = 'vagrant'

  client.vm.network :private_network,
                    type: "dhcp",
                    virtualbox__intnet: true

  client.vm.provider :virtualbox do |virtualbox|
    virtualbox.customize ["modifyvm", :id, "--vrd", "on"]
    virtualbox.customize ["modifyvm", :id, "--vrdeport", "3392"]
  end

  client.vm.provision "client dummy",
                    type: "shell",
                    preserve_order: true,
                    path: "provision/client/01-dummy.sh"

  client.vm.provision "client routing",
                    type: "shell",
                    preserve_order: true,
                    run: "always",
                    path: "provision/client/01-routing.sh"

end
end

```

1.5.1.4. Содержание файла Makefile для packer

Цели по умолчанию

```
.DEFAULT_GOAL := help

.PHONY: version

# Получение текущего каталога
CURDIR := $(realpath .)

init: ## Install missing plugins for packer
    @mkdir -p "$(CURDIR)"/.config/packer/plugins
    @export PACKER_CONFIG_DIR="$(CURDIR)"/.config/packer; export
    ↪ PACKER_PLUGIN_PATH="$(CURDIR)"/.config/packer/plugins; packer
    ↪ init vagrant-rocky.pkr.hcl

virtualbox: init ## Build Rocky Linux box for Virtualbox
    -@VBoxManage setproperty language C
    -@VBoxManage setproperty machinefolder "$(CURDIR)/vm
    -@export TMPDIR="$(CURDIR)"; export
    ↪ PACKER_CONFIG_DIR="$(CURDIR)"/.config/packer; export
    ↪ PACKER_PLUGIN_PATH="$(CURDIR)"/.config/packer/plugins; packer
    ↪ build -only=virtualbox-iso.rockylinux vagrant-rocky.pkr.hcl
    -@VBoxManage setproperty machinefolder default

qemu: init ## Build Rocky Linux box for Qemu
    -@export TMPDIR="$(CURDIR)"; export
    ↪ PACKER_CONFIG_DIR="$(CURDIR)"/.config/packer; export
    ↪ PACKER_PLUGIN_PATH="$(CURDIR)"/.config/packer/plugins; packer
    ↪ build -only=qemu.rockylinux vagrant-rocky.pkr.hcl

help:
    @echo 'Usage:'
    @echo '  make <target>'
    @echo
    @echo 'Targets:'
    @grep -E '^[a-zA-Z_0-9.-]+:.*?## .*$$' $(MAKEFILE_LIST) | sort | awk
    ↪ 'BEGIN {FS = ":.*?## "}; {printf "  \033[36m%-30s\033[0m %s\n",
    ↪ $1, $$2}'
    @echo
```

1.5.1.5. Содержание файла Makefile для vagrant

```
## Конфигурация
BOX_NAME := rockylinux10
VAGRANT_FILE ?= Vagrantfile
## Set your provider
# PROVIDERS := virtualbox libvirt
PROVIDERS := virtualbox

# Цели по умолчанию
.DEFAULT_GOAL := help

.PHONY: version

# Получение текущего каталога
CURDIR := $(realpath .)

help:
    @echo 'Usage:'
    @echo '  make <target>'
```

```

@echo
@echo 'Targets:'
@grep -E '^[a-zA-Z_0-9-]+:.*?## .*$$' $(MAKEFILE_LIST) | sort | awk
  ↪ 'BEGIN {FS = ":.*?## "}; {printf " \033[36m%-30s\033[0m %s\n",
  ↪ $1, $2}'
@echo

check-virtualbox: ## Check VirtualBox
@which VBoxManage >/dev/null 2>&1 || (echo "VirtualBox is not
  ↪ installed"; exit 1)

check-libvirt: ## Check libvirt
@which virsh >/dev/null 2>&1 || (echo "libvirt is not installed";
  ↪ exit 1)
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
  ↪ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; vagrant plugin list |
  ↪ grep -q vagrant-libvirt || (echo "The vagrant-libvirt plugin is
  ↪ not installed"; exit 1)

plugins: ## Install plugins
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
  ↪ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; vagrant plugin install
  ↪ --plugin-clean-sources --plugin-source https://rubygems.org
  ↪ vagrant-libvirt
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
  ↪ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; vagrant plugin install
  ↪ --plugin-clean-sources --plugin-source https://rubygems.org
  ↪ vagrant-vbguest
touch plugins

addbox: plugins ## Add the built box to Vagrant
@for provider in $(PROVIDERS); do \
  export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
  ↪ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; vagrant box
  ↪ add $(BOX_NAME) vagrant-$$provider-$(BOX_NAME)-x86_64.box
  ↪ ; \
  if [[ "$$provider" == "libvirt" ]] then mkdir -p vm ; virsh
  ↪ pool-define-as --name vagrant --type dir --target
  ↪ "$$(CURDIR)/vm ; virsh pool-start vagrant ; fi ; \
done

server-up: plugins ## Start server
@VBoxManage setproperty machinefolder "$(CURDIR)/vm
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
  ↪ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; export
  ↪ VBox_USER_HOME="$(CURDIR)/.vbox; export
  ↪ VBox_INSTALL_PATH="$(CURDIR)/vm; vagrant up
  ↪ --provider=virtualbox server
@VBoxManage setproperty machinefolder default

client-up: plugins ## Start client
@VBoxManage setproperty machinefolder "$(CURDIR)/vm
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
  ↪ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; export
  ↪ VBox_USER_HOME="$(CURDIR)/.vbox; export
  ↪ VBox_INSTALL_PATH="$(CURDIR)/vm; vagrant up
  ↪ --provider=virtualbox client

```

```

@VBoxManage setproperty machinefolder default

server-ssh:      ## Start server
@VBoxManage setproperty machinefolder "$(CURDIR)/vm
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
↳ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; export
↳ VBox_USER_HOME="$(CURDIR)/.vbox; export
↳ VBox_INSTALL_PATH="$(CURDIR)/vm; vagrant ssh server
@VBoxManage setproperty machinefolder default

client-ssh:      ## Start client
@VBoxManage setproperty machinefolder "$(CURDIR)/vm
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
↳ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; export
↳ VBox_USER_HOME="$(CURDIR)/.vbox; export
↳ VBox_INSTALL_PATH="$(CURDIR)/vm; vagrant ssh client
@VBoxManage setproperty machinefolder default

server-halt: plugins      ## Stop server
@VBoxManage setproperty machinefolder "$(CURDIR)/vm
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
↳ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; export
↳ VBox_USER_HOME="$(CURDIR)/.vbox; export
↳ VBox_INSTALL_PATH="$(CURDIR)/vm; vagrant halt server
@VBoxManage setproperty machinefolder default

client-halt: plugins      ## Stop client
@VBoxManage setproperty machinefolder "$(CURDIR)/vm
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
↳ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; export
↳ VBox_USER_HOME="$(CURDIR)/.vbox; export
↳ VBox_INSTALL_PATH="$(CURDIR)/vm; vagrant halt client
@VBoxManage setproperty machinefolder default

server-provision: plugins      ## Start and provision server
@for provider in $(PROVIDERS); do \
    VBoxManage setproperty machinefolder "$(CURDIR)/vm; \
    export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
    ↳ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; export
    ↳ VBox_USER_HOME="$(CURDIR)/.vbox; export
    ↳ VBox_INSTALL_PATH="$(CURDIR)/vm; vagrant up
    ↳ --provider=$$provider server --provision ; \
    VBoxManage setproperty machinefolder default ; \
done

client-provision: plugins      ## Start and provision client
@for provider in $(PROVIDERS); do \
    VBoxManage setproperty machinefolder "$(CURDIR)/vm ; \
    export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
    ↳ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; export
    ↳ VBox_USER_HOME="$(CURDIR)/.vbox; export
    ↳ VBox_INSTALL_PATH="$(CURDIR)/vm; vagrant up
    ↳ --provider=$$provider client --provision ; \
    VBoxManage setproperty machinefolder default ; \
done

server-destroy:      ## Destroy server

```

```

@VBoxManage setproperty machinefolder "$(CURDIR)/vm
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
  ↪ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; export
  ↪ VBOX_USER_HOME="$(CURDIR)/.vbox; export
  ↪ VBOX_INSTALL_PATH="$(CURDIR)/vm; vagrant destroy -f server
@VBoxManage setproperty machinefolder default

client-destroy:    ## Destroy client
@VBoxManage setproperty machinefolder "$(CURDIR)/vm
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
  ↪ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; export
  ↪ VBOX_USER_HOME="$(CURDIR)/.vbox; export
  ↪ VBOX_INSTALL_PATH="$(CURDIR)/vm; vagrant destroy -f client
@VBoxManage setproperty machinefolder default

status: plugins  ## Show status of all VMs
@for provider in $(PROVIDERS); do \
  echo "=== Provider: $$provider ==="; \
  export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
  ↪ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; vagrant status
  ↪ --machine-readable --provider=$$provider | awk -F, '$$2 ==
  ↪ "default" && $$3 == "state" {print $$4}'; \
done

destroy: server-destroy client-destroy  ## Destroy all VM

clean: destroy  ## Full clean
rm -rf .vagrant
rm -f plugins
@export VAGRANT_HOME="$(CURDIR)/.vagrant.d; export
  ↪ VAGRANT_DOTFILE_PATH="$(CURDIR)/.vagrant; export
  ↪ VBOX_USER_HOME="$(CURDIR)/.vbox; export
  ↪ VBOX_INSTALL_PATH="$(CURDIR)/vm; vagrant box remove $(BOX_NAME)
  ↪ --all --force || true
@for provider in $(PROVIDERS); do \
  if [[ "$$provider" == "libvirt" ]] ; then virsh pool-delete
  ↪ vagrant ; virsh pool-undefine vagrant ; fi ; \
done

```

1.5.2. Развёртывание лабораторного стенда на ОС Linux

1. Перейдите в каталог с проектом:
`cd /var/tmp/user_name/packer`
 где `user_name` — идентифицирующее вас имя пользователя, обычно первые буквы инициалов и фамилия.
2. В терминале наберите
`make help`
3. Вы увидите перечень указанных в Makefile целей и краткое описание их действий. Для формирования box-файла с дистрибутивом Rocky Linux для VirtualBox в терминале наберите
`make init`
`make box`

Начнётся процесс скачивания, распаковки и установки драйверов VirtualBox и дистрибутива ОС на виртуальную машину. Во время автоматического развёртывания дистрибутива можно просматривать выводимую на экран информацию с разных окон, перемещаясь по ним с помощью клавиш `Alt` + `Tab`.

После завершения процесса автоматического развёртывания образа виртуальной машины в каталоге `/var/tmp/user_name/packer` временно появится каталог `builds` с промежуточными файлами `.vdi`, `.vmdk` и `.ovi`, которые затем автоматически будут преобразованы в `box`-файл сформированного образа: `vagrant-virtualbox-rockylinux10-x86_64.box`.

4. Сохраните файлы `vagrant-virtualbox-rockylinux10-x86_64.box`, `vagrant-rocky.pkr.hcl`, `./http/ks.cfg`, `Vagrantfile` и `Makefile` на внешний носитель или другой каталог.
5. Скопируйте или переместите файл `vagrant-virtualbox-rockylinux10-x86_64.box` в рабочий каталог в подкаталог `vagrant`.
6. Для регистрации образа виртуальной машины в Vagrant в терминале в каталоге `/var/tmp/user_name/vagrant` наберите
`make addbox`

Это позволит на основе конфигурации, прописанной в файле `Vagrantfile`, сформировать `box`-файлы образов двух виртуальных машин — сервера и клиента с возможностью их параллельной или индивидуальной работы.

7. Запустите виртуальную машину `Server`, введя
`make server-up`
8. Запустите виртуальную машину `Client`, введя
`make client-up`
9. Подключитесь к серверу из консоли:
`vagrant ssh server`
10. Введите пароль `vagrant`.
11. Перейдите к пользователю `user` (вместо `user` должен быть указан ваш логин):
`su - user`

В случае появления сообщения

```
su: user user does not exist or the user entry does not contain all the
    ^ required fields
```

необходимо выполнить действия в консоли основной ОС:

- отлогиниться из виртуальной машины: `logout` или `exit`, или комбинация клавиш `Ctrl`+`d`;
- выключить виртуальную машину:
`make server-halt`
- запустить еще раз виртуальную машину, убедившись в отработке скриптов `hostname` и `user`;
- подключиться к серверу из консоли и перейти под пользователя `user` (вместо `user` должен быть указан ваш логин).

12. Отлогиньтесь.
13. Выполните тоже самое для клиента.
14. Выключите обе виртуальные машины:
`make server-halt`
`make client-halt`
15. Убедитесь, что запуск обеих виртуальных машин прошёл успешно, залогиньтесь под пользователем `vagrant` с паролем `vagrant` в графическом окружении.

1.5.3. Развёртывание лабораторного стенда на ОС Windows

В данном разделе приведена последовательность действий при развёртывании образа виртуальной машины в ОС Windows на домашнем компьютере в VirtualBox с использованием Vagrant. После установки необходимого программного обеспечения не забудьте перезагрузить систему.

Далее выполните следующие действия:

1. Используя FAR, перейдите в созданный вами рабочий каталог с проектом. В этом же каталоге должен быть размещён файл `packer.exe`. В командной строке введите
`packer.exe init vagrant-rocky.pkr.hcl`
`packer.exe build vagrant-rocky.pkr.hcl`

для начала автоматической установки образа операционной системы Rocky Linux в VirtualBox и последующего формирования box-файла с дистрибутивом Rocky Linux для VirtualBox. По окончании процесса в рабочем каталоге сформируется box-файл с названием `vagrant-virtualbox-rockylinux10-x86_64.box`.

2. Для регистрации образа виртуальной машины в `vagrant` в командной строке введите
`vagrant box add rockylinux10 vagrant-virtualbox-rockylinux10-x86_64.box`
3. Для запуска виртуальной машины `Server` введите в консоли
`vagrant up server`
4. Для запуска виртуальной машины `Client` введите в консоли
`vagrant up client`
5. Убедитесь, что запуск обеих виртуальных машин прошёл успешно, залогиньтесь под пользователем `vagrant` с паролем `vagrant` в графическом окружении.
6. Подключитесь к серверу из консоли:
`vagrant ssh server`
7. Введите пароль `vagrant`.
8. Перейдите к пользователю `user` (вместо `user` должен быть указан ваш логин):
`su - user`
9. Отлогиньтесь.
10. Выполните тоже самое для клиента.
11. Выключите обе виртуальные машины:
`vagrant halt server`
`vagrant halt client`

1.5.4. Внесение изменений в настройки внутреннего окружения виртуальной машины

1. Для отработки созданных скриптов во время загрузки виртуальных машин убедитесь, что в конфигурационном файле `Vagrantfile` до строк с конфигурацией сервера имеется следующая запись:

```
# Common configuration
config.vm.provision "common user",
  type: "shell",
  preserve_order: true,
  path: "provision/default/01-user.sh"

config.vm.provision "common hostname",
  type: "shell",
  preserve_order: true,
  run: "always",
  path: "provision/default/01-hostname.sh"
```

2. Зафиксируйте внесённые изменения для внутренних настроек виртуальных машин, введя в терминале:

```
make server-provision
Затем
make client-provision
```

Для работающих под ОС Windows вместо инструкций `Makefile` следует последовательно ввести в командной строке:

```
vagrant up server --provision
vagrant up client --provision
```

3. Залогиньтесь на сервере и клиенте под созданным пользователем. Убедитесь, что в терминале приглашение отображается в виде `user@server.user.net` на сервере и в виде `user@client.user.net` на клиенте, где вместо `user` указан ваш логин.
4. Выключите виртуальные машины.

5. После выключения виртуальных машин скопируйте необходимые для работы с Vagrant файлы и box-файлы виртуальных машин на внешний носитель или в другой каталог вашей ОС. Используя эти файлы, вы можете развернуть виртуальные машины на другом компьютере.

1.6. Содержание отчёта

1. Титульный лист с указанием номера лабораторной работы и ФИО студента.
2. Формулировка задания работы.
3. Описание результатов выполнения задания:
 - скриншоты (снимки экрана), фиксирующие выполнение работы;
 - подробное описание настроек служб в соответствии с заданием;
 - полные тексты конфигурационных файлов настраиваемых в работе служб;
 - результаты проверки корректности настроек служб в соответствии с заданием (подтверждённые скриншотами).
4. Выводы, согласованные с заданием работы.
5. Ответы на контрольные вопросы.

1.7. Контрольные вопросы

1. Для чего предназначен Vagrant?
2. Что такое box-файл? В чём назначение Vagrantfile?
3. Приведите описание и примеры вызова основных команд Vagrant.
4. Дайте построчные пояснения содержания файлов `vagrant-rocky.pkr.hcl`, `ks.cfg`, `Vagrantfile`, `Makefile`.

При ответах на вопросы рекомендуется ознакомиться с источниками [1–5].

Список литературы

1. GNU Bash Manual. — 2019. — URL: <https://www.gnu.org/software/bash/manual/> (дата обр. 13.09.2021).
2. GNU Make Manual. — 2016. — URL: <http://www.gnu.org/software/make/manual/> (visited on 09/13/2021).
3. Powers S. Vagrant Simplified [Построю Vagrant] / Пер.: А. Панин // Библиотека сайта rus-linux.net. — 2015. — URL: <http://rus-linux.net/MyLDP/vm/vagrant-simplified.html> (visited on 09/13/2021).
4. Vagrant Documentation. — URL: <https://www.vagrantup.com/docs> (visited on 09/13/2021).
5. Купер М. Искусство программирования на языке сценариев командной оболочки. — 2004. — URL: https://www.opennet.ru/docs/RUS/bash_scripting_guide/ (дата обр. 13.09.2021).